Recreating Our Claude + OpenGov Workflows

Sep 24, 2026 · @Dan

Each section below is a prompt you can paste into Claude to rebuild one of the City of Hilliard's OpenGov workflows for your own organization. Replace everything in [brackets] with your details.

Before you start

You need four things before the first prompt:

1. **Sign-off from IT** on your organization's AI and data policy. The browser workflows act with your own OpenGov permissions; the chat agent uses an API key.
2. **The Claude desktop app** with Cowork, on a paid Claude plan.
3. **The Claude in Chrome extension**, installed in the Chrome profile where you are signed in to OpenGov Permitting & Licensing.
4. **Your OpenGov report URLs.** Most workflows start from a saved report. Build or find the report first and copy its link.

How to use these prompts:

- Start a new Cowork task, paste the prompt, and answer Claude's questions.
- Run the first pass on 1 record, then 10, while you watch. Correct anything it gets wrong in plain language.
- When a run is right, finish with the "save as a skill" prompt at the end of this guide. After that the whole workflow is one sentence, like "close the next 25."
- Attach any template (Excel, PDF, Word) the output has to match. Claude copies its layout instead of inventing one.

1. Month-end reports and daily reconciliation

These two replace export-and-retype work. Attach last month's finished packet or last day's filled template so Claude can match it.

Month-end packet prompt:

```
Every month our [division name] sends a month-end report packet built from
OpenGov Permitting & Licensing data. I've attached last month's finished
packet and our yearly workbook.

Using the Claude in Chrome extension (I'm signed in to [your OpenGov URL]):
1. Open the report [report URL] and pull last month's rows. If you can, read
the data with the page's own API calls instead of clicking Export.
2. Tell me the row count and dollar totals before you build anything, so I
can check them.
3. Fill the [month] tab of the workbook, then build each form in the packet
([list forms, e.g. Dodge report, Census C-404, state 1%/3% surcharge forms])
in the same layout as the attached example.
4. Combine everything into one PDF in the packet's order.
5. Draft the distribution emails (recipients: [list]). Do NOT send email. I
will send them myself.

Never save changes to a shared OpenGov report. If you change a filter, click
Discard Changes.
```

Daily reconciliation prompt:

```
Each business day I reconcile the prior day's OpenGov deposits into the
attached Excel template.

Using Claude in Chrome:
1. Open [daily deposit summary report URL] and set the Date Paid filter to
[date].
2. Export the report and read the CSV.
3. Fill the template the same way the attached example is filled.
4. Check that the payment-method total equals the account total. If they
don't tie, stop and show me the difference and the records involved.
5. Save the file as [naming pattern, e.g. YYYY.MM.DD Reconciliation.xlsx].
Draft the email to [recipients] but don't send it.
```

2. Permit activity dashboard

This builds a web page your team can reopen each day. Claude publishes it as a private page you can share.

```
Build me a dashboard of OpenGov permit activity. Using Claude in Chrome
(signed in to [your OpenGov URL]), pull from these reports: [report URLs].

The dashboard should show:
- Permits closed or completed each day for the last [30/90] days, by record
type
- The number of related records on each permit
- A list of permits still Active whose record was created more than [3] years
ago, with record number, type, address and a link to the record

Publish it as a page I can reopen and share. Tell me which numbers you could
not get and why, rather than estimating them. When it's done, write down the
steps so I can ask you to refresh it with one sentence.
```

For a public-facing version (for example, new housing units permitted by year), say so in the prompt and ask for only non-personal, aggregate data on the page.

3. Weekly fee audit

Claude compares every fee paid in a week to your adopted fee schedule and to the record itself, then lists the exceptions. Paste in your fee schedule or give Claude the link to it.

```
Run an internal audit of fees collected in OpenGov Permitting & Licensing for [date range, default: last Monday through Sunday, by Date Paid].

Our adopted fee schedule is here: [link to code of ordinances / attached fee schedule]. Focus on [fee types, e.g. water and sewer capacity fees, impact fees, residential building permit fees, commercial plan review fees].

Using Claude in Chrome:
1. Open the Ledger report under Payments ([report URL]) and capture every row paid in the date range.
2. For any fee that depends on the record (number of units, meter size, square footage, location), open the record and read the value.
3. Check each fee against the schedule. Flag: amounts that match no tier, fees charged on the wrong record type, required fees that are missing, and duplicate payments.
4. Give me an Excel file with one row per exception: record number and link, fee label, amount paid, expected amount, and the reason.

Do not change any record or report. This is read-only. If a rate may have changed, tell me which ordinance to check instead of assuming.
```

4. Closing stale Active permits

This clears old permits that sit Active only because nobody finished the workflow. The key rule: close only records waiting on the final close step, and skip everything else.

```
I need to administratively close old permits in OpenGov Permitting &
Licensing. Work from this report, oldest first: ["Active since" report URL].

Using Claude in Chrome, for each record:
1. Open the record's Workflow tab.
2. Check that the status is Active, the only step In Progress is named
exactly "[Close Permit]", and nothing else is in progress.
3. If anything else is in progress (an inspection, a review, a hold) or the
record is already Complete, SKIP it and add it to a skipped list. Do not
close it.
4. Otherwise open the [Close Permit] step, post an INTERNAL note: "[Closing
permit administratively]", then click Complete Step.
5. Re-read the record to confirm the status now shows Complete and the note
saved.

Read all the record numbers from the report once at the start. Show me the
first record (number, address, note) and wait for my OK. Then do the next
[10] without stopping, and finish with a table: closed, skipped (with the
reason), and errors.
```

Lessons from our runs: click the note box by its page element, not by screen position, or the note may not save. Wait a couple of seconds between typing and clicking Send.

5. Other record cleanup

The same confirm-first, skip-when-unsure pattern works for other cleanup. Swap the middle of the prompt above for one of these:

Task	What to tell Claude
Archive duplicates	"Work down [report URL]. For each address with more than one [record type] record, keep the first one submitted and archive the others. Show me each group before archiving."
Fix placeholder addresses	"Work down the attached spreadsheet of Record # and correct address. Set the Primary Location only when OpenGov has an exact match. Never guess; list the ones with no match."

Task	What to tell Claude
Attach scanned paper permits	"The PDFs in [folder] are named by permit number. Upload each one to its matching OpenGov record and mark it done in the attached tracking sheet."
Trade permits blocking a building permit	"For each building permit in [report URL], find the active [HVAC / electrical / gas] permits at the same address submitted within [6] months. Relate them and add a workflow blocker on [step name]. Skip completed and temporary permits."

For anything that deletes or archives, ask Claude to show you the full list first and approve it before it acts.

6. Chat agent connected to the OpenGov API

The chat agent reads OpenGov through its official API, so it works for anyone at any time without a staff browser open. Claude writes all the code; you supply keys and test.

```
flowchart LR
  A[Chat web page<br/>no keys here] --> B[Small server / Worker<br/>holds keys, checks staff password]
  B --> C[Claude API<br/>understands, picks a tool]
  B --> D[OpenGov PLCE API v2<br/>reads and writes records]
  B --> E[Other sources<br/>GIS, agendas, programs]
```

The page never holds a key. Every OpenGov and Claude call goes through the server.

Set up the OpenGov connection:

1. In the OpenGov Developer Portal (developer.opengov.com), go to Integrations and create an API key for your community.
2. In the OpenGov employee portal, turn on API access for each record type the agent should read or create.
3. Get a Claude API key from the Claude Console (console.anthropic.com).
4. Create a free Cloudflare account (or use your own server) and store both keys as secrets there, never in the web page or a code repository.
5. Grant the least access you need. Hilliard's key cannot delete or archive records.

The API calls the agent uses:

Purpose	Call
Find an address	`GET /locations?filter[streetNumber]=…&filter[streetName]=…`
Permits at an address	`GET /records?filter[locationID]=…` (add `filter[recordTypeID]` to narrow)
Create a record	`POST /records` with the record type ID (starts as Draft)
Set its address	`PATCH /records/{id}/primary-location`
Attach a file	`POST /files`, upload the bytes, then `POST /records/{id}/attachments`
Submit it	`PATCH /records/{id}` with status ACTIVE

Base URL: `https://api.plce.opengov.com/plce/v2/[your community ID]`, header `Authorization: Token [key]`. Filling a record's form fields through the API needs OpenGov's newer Forms version; ask your OpenGov representative whether it is enabled for your community.

Build prompt:

```
Help me build a chat assistant for [organization] residents and staff. Front end: a single web page I can host on [GitHub Pages / our website]. Back end: a Cloudflare Worker that stores all API keys as secrets.

The assistant uses Claude with tools. Start with these tools:
- lookup_permits: given an address, find the OpenGov location with the PLCE API v2 (base URL [URL], key in a secret) and list its records with type, status and a public link.
- lookup_zoning: given an address, query our GIS service [ArcGIS REST URL] for zoning district.
- [other sources: project list CSV, meeting agendas, rec programs]

Add a staff mode: a checkbox and a password. The Worker must check the password on the server on every request before allowing any staff tool.

Rules for the assistant: answer only from tool results, link to the source record, and say when it doesn't know. Give me the files one at a time with instructions for where each goes, and a test plan I can run before going live.
```

7. Code enforcement letters that open an OpenGov record

Once the chat agent works, add this staff-only feature. Attach one real letter you've sent as the model, with personal details removed.

```
Add a staff-only code enforcement letter feature to our chat assistant. I've
attached an example letter.

Flow:
1. Staff enters a property address and picks the violation type ([e.g. tall
grass and weeds]).
2. Look up the owner name and MAILING address from [county auditor GIS
service URL]. Use the owner's mailing address, not the tax-bill address.
3. Ask staff which inspector is sending it: [names, titles, phone, email].
4. Draft the letter in the attached format, citing [code section and title,
e.g. 917.02 Removal of Weeds]. Anything the tools didn't return goes in as a
highlighted [bracketed blank]. Never invent names, parcel numbers or dates.
5. Show the letter in the chat with a Word download.
6. Only after staff clicks Create Record: create an OpenGov record of type
[record type ID] with the PLCE API (Draft), set its primary location to the
property, attach the letter, then submit it. Show a link to the new record.

This tool must require the staff password, checked on the server. Build it so
I can test it on a test address before going live.
```

Find your record type ID in OpenGov, or ask Claude to look it up with the API once your key is set up.

8. Save it as a skill, and the guardrails

When a workflow runs correctly, save it so the next run is one sentence. Paste this at the end of a good session:

```
That run was right. Save this whole process as a skill named [short-name], so
next time I can just say "[trigger phrase, e.g. close the next 25 permits]".
Include:
- the report URLs and default settings we used
- every rule I gave you, including the corrections I made during this session
- the checks that decide when to skip a record
- what to show me before you change anything
- the interaction tricks that made the browser steps reliable
Show me the skill before saving.
```

Guardrails we use on every workflow:

- **Confirm first.** Claude shows the first record and waits for an OK before a batch.
- **Skip, don't guess.** Anything ambiguous goes on a list for a person.

- **Blanks, not inventions.** Missing names, amounts or dates stay as [bracketed blanks].
- **Drafts only for email.** A person sends every message.
- **No passwords from Claude.** You sign in; Claude never types a password or key.
- **Read-only reports.** Never save changes to shared OpenGov reports.
- **Verify after saving.** Claude reopens the record to confirm the change stuck.
- **Small batches.** Start with 10. Browser sessions can drop mid-run; typing "continue" resumes.